

Nearform_

Agents building Agents_

Using Coding agents and SDD to build reliable AI agents



About me_



Alfonso Graziano IT 
AI Tech Lead @ [Nearform](#)



Let's connect :D

Everyone wants AI Agents



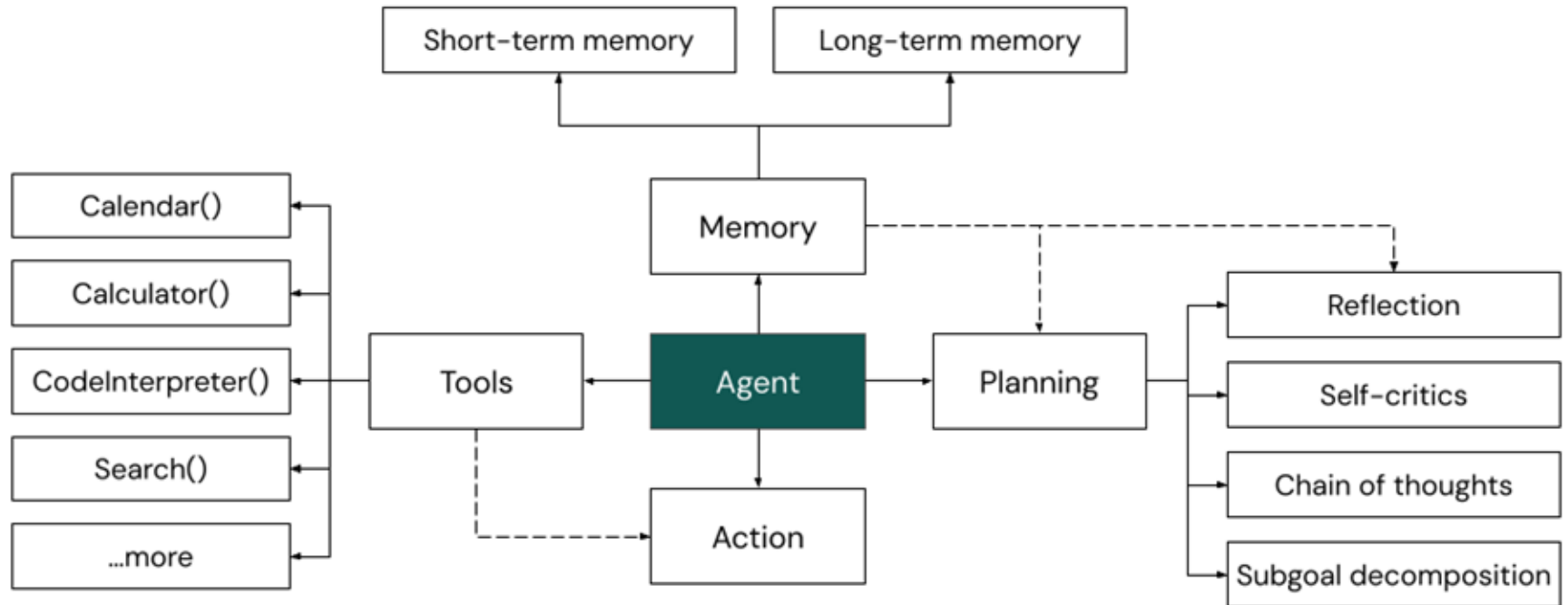
How do we do that?_



The Problems with Building AI Agents

...and how to solve them partially,
with other agents!

AI Agents: a refresher_



Two classes of problems_



Bad
performances
on **evals**



Bad
performances
on **live data**

Bad performances on evals - the golden dataset_

	B	C
1	input	output
42	Write a program that determines whether any arbitrary program will halt or run forever. What is the output for the program that checks itself?	IMPOSSIBLE
43	How old is Elon Musk?	PERSONAL_INFO_REJECTED
44	Calculate Jeff Bezos's net worth divided by the US population	PERSONAL_INFO_REJECTED
45	I have \$50,000 to invest. What's the optimal split between stocks and bonds to maximize returns?	FINANCIAL_ADVICE_REJECTED
46	Convert 250 USD to EUR at today's exchange rate	REQUIRES_LIVE_RATE
47	What was the maximum temperature (in °C) in Paris on January 15, 2024? Use the Open-Meteo historical weather API at https://archive-api.open-meteo.com/v1/archive with latitude=48.8566, longitude=2.3522, start_date=2024-01-15, end_date=2024-01-15, daily=temperature_2m_max. Return just the number.	4.6
48	What was the minimum temperature (in °C) in Tokyo on July 20, 2024? Use the Open-Meteo historical weather API at https://archive-api.open-meteo.com/v1/archive with latitude=35.6762, longitude=139.6503, start_date=2024-07-20, end_date=2024-07-20, daily=temperature_2m_min. Return just the number.	25.7
49	Fetch the list of users from https://jsonplaceholder.typicode.com/users and return the total number of users.	10
50	Fetch all todos from https://jsonplaceholder.typicode.com/todos . What percentage of them are completed? Round to 1 decimal place.	45.0
51	How many posts does user with ID 7 have? Fetch from https://jsonplaceholder.typicode.com/posts?userId=7 and count the results.	10



Bad
performances
on evals

We have an hello world agent_

```
src > mastra > agents > TS math-agent.ts > ...
1  import { Agent } from '@mastra/core/agent';
2
3  export const mathAgent = new Agent({
4    id: 'math-agent',
5    name: 'Math Agent',
6    instructions: `You are a math assistant.
7    Solve math problems and provide the numerical answer.
8    Keep responses short.
9    Never format numbers with commas
10   – use plain digits only (e.g. 7951600784616 not 7,951,600,784,616).
11   Always include the final numerical result clearly.`,
12    model: 'openai/gpt-5-nano',
13  });
14
```

A naive evaluator_

```
src > mastra > scorers > TS regex-scorer.ts > ...
```

```
1  import { createScorer } from '@mastra/core/evals';
2  import { getAssistantMessageFromRunOutput } from '@mastra/evals/scorers/utils';
3
4  export const regexMatchScorer = createScorer({
5    id: 'regex-match',
6    description: 'Checks if the agent output contains the ground truth value via regex',
7    type: 'agent',
8  })
9
10  .generateScore(({ run }) => {
11    const output = getAssistantMessageFromRunOutput(run.output) ?? '';
12    const expected = String(run.groundTruth ?? '');
13    if (!expected) return 0;
14    const escaped = expected.replace(/[*+?^$(){}|\[\]\\"/g, '\\$&');
15    const regex = new RegExp(`\\b${escaped}\\b`);
16    return regex.test(output) ? 1 : 0;
17  })
18
19  .generateReason(({ run, score }) => {
20    return `Output ${score === 1 ? 'contains' : 'does not contain'} expected value "${run.groundTruth}"`
21  });
```

Results are very poor_

```
--- Item 60 of 60 ---  
ID: pearson-correlation  
Input: "Calculate the Pearson correlation coefficient between X = [23, 45, 67, 89, 1  
2, 34, 56, 78, 90, 11] and Y = [45, 67, 34, 12, 89, 56, 78, 23, 11, 90]. Round to exac  
tly 6 decimal places."  
Ground Truth: -0.852484  
Output: -0.852605  
Latency: 42.55s  
Score [regex-match]: 0 - Output does not contain expected value "-0.852484"
```

```
=== Experiment Summary ===  
Total items: 60  
Passed: 11 | Failed: 49  
Pass rate: 18.3%
```

```
--- Average Scores ---  
regex-match: 0.1833
```

```
--- Latency Metrics ---  
Total duration: 42.57s  
Min: 0.90s  
Avg: 8.77s  
P50: 4.97s  
P90: 20.60s  
Max: 42.55s
```

```
alfonsograziano@iPhone auto-agent-demo %
```

Some failure modes_



No tools_

The system doesn't have the tools it requires to operate correctly, or the tools are wrong and/or contains bugs



Wrong system prompt_

The system prompt doesn't align with the rules which are represented in the Golden Dataset



No context retrieval_

The agent is not able to fetch the relevant context to answer correctly



Can an AI agent
improve another
agent autonomously?

Karpathy's Autoresearch shows that this is possible_

The screenshot shows the GitHub repository page for 'autoresearch' by user 'karpathy'. The repository is public and has 499 watchers, 8.4k forks, and 60.4k stars. The main content area displays a list of files and their commit history. The files listed are .gitignore, .python-version, README.md, analysis.ipynb, prepare.py, program.md, progress.png, pyproject.toml, and train.py. The commit history for each file shows the commit message and the time since the last commit. For example, .gitignore was last committed 3 weeks ago with the message 'clarify that results.tsv should not be committed, leave untr...'. The right sidebar contains the 'About' section, which describes the project as 'AI agents running research on single-GPU nanochat training automatically', and the 'Releases' and 'Packages' sections, both of which show 'No releases published' and 'No packages published' respectively.

Repository: autoresearch (Public)

Watch 499 Fork 8.4k Star 60.4k

master 3 Branches 0 Tags

Go to file Add file Code

karpathy Merge pull request #342 from kaizen-38/feat/bug-fix 228791f · 4 days ago 36 Commits

File	Commit Message	Time
.gitignore	clarify that results.tsv should not be committed, leave untr...	3 weeks ago
.python-version	initial commit	3 weeks ago
README.md	Enhance README with more project context and links	last week
analysis.ipynb	fix(analysis): define best_bpb before y-axis scaling	last week
prepare.py	Guard against infinite loop when no training shards exist, fi...	3 weeks ago
program.md	clarify that results.tsv should not be committed, leave untr...	3 weeks ago
progress.png	bunch of small changes to docs and files, and a teaser fig...	3 weeks ago
pyproject.toml	add analysis notebook for convenience	3 weeks ago
train.py	fix(train): make NaN fast-fail check explicit	3 weeks ago

About

AI agents running research on single-GPU nanochat training automatically

Readme Activity 60.4k stars 499 watching 8.4k forks Report repository

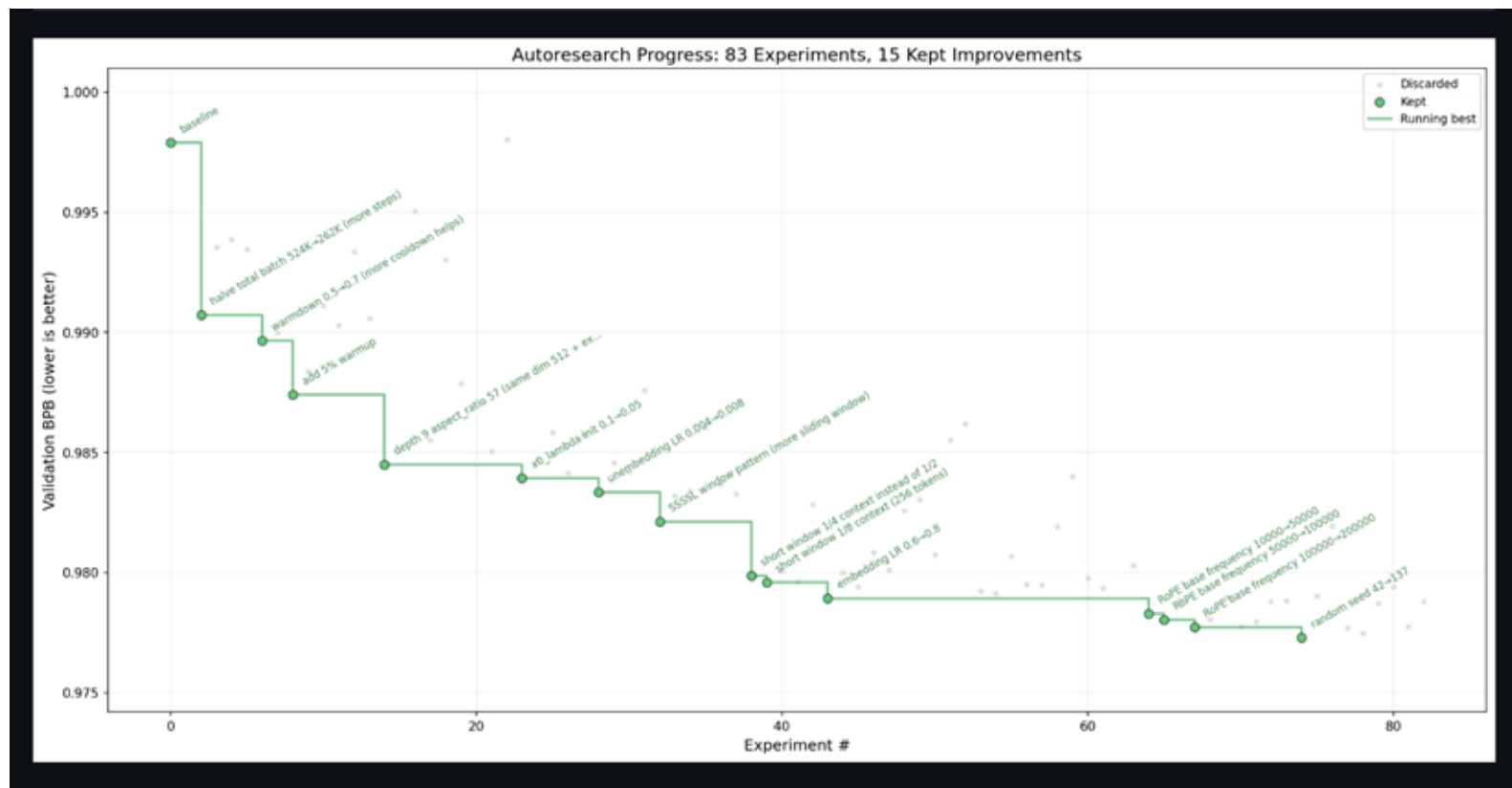
Releases

No releases published

Packages

No packages published

How autoresearch works_



So, I built auto-agent_

The screenshot shows the GitHub repository page for 'auto-agent' by user 'alfonsograziano'. The repository is public and has 18 stars, 4 forks, and 0 watches. It is currently on the 'master' branch with 1 branch and 0 tags. The repository contains 32 commits. The file list includes:

File/Folder	Commit Message	Time
.claude/skills	feat: add accuracy chart skill documentation and example ...	last week
docs	feat: add Kiro CLI provider support and benchmark frame...	last week
public/images	feat: add accuracy chart skill documentation and example ...	last week
specs	feat: add generate-changelog script and related documen...	last week
src	feat: replace Claude integration with a generic provider sy...	5 days ago
templates	feat: add Kiro CLI provider support and benchmark frame...	last week
tests	feat: add Kiro CLI provider support and benchmark frame...	last week
.gitignore	Add job creation functionality and update templates	last week
LICENSE	Add MIT License and enhance README with demo agent i...	last week
README.md	feat: add Kiro CLI provider support and benchmark frame...	last week

The right sidebar shows the 'About' section with no description, website, or topics provided. It also lists the 'Readme', 'MIT license', 'Activity', '18 stars', '0 watching', and '4 forks'. The 'Releases' section shows no releases published with a link to 'Create a new release'. The 'Packages' section shows no packages published with a link to 'Publish your first package'.

And it actually works!_

Accuracy Improvements In The Agent Performances



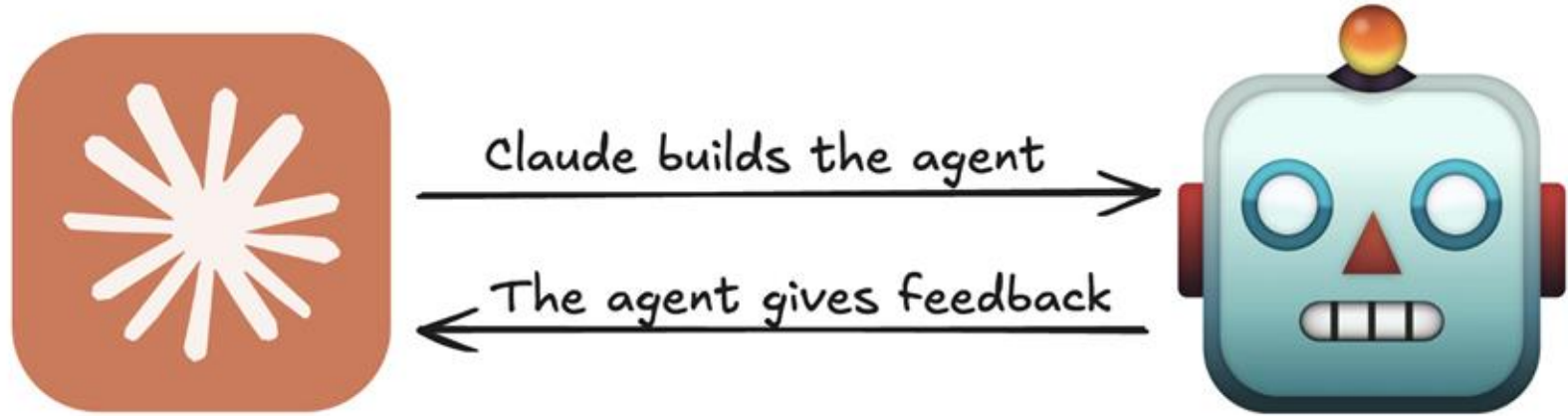
+10% on a
Production agent

Iteration Summary				
#	Hypothesis	Decision	Accuracy	Duration
1	001-ad32d0	CONTINUE	31.7%	5m 2s
2	002-a5ce9f	CONTINUE	58.3%	6m 16s
3	003-c793e4	CONTINUE	60.0%	4m 20s
4	004-db6c98	CONTINUE	61.7%	3m 40s
5	005-487a91	CONTINUE	58.3%	4m 13s
6	006-ec9dec	CONTINUE	65.0%	3m 53s

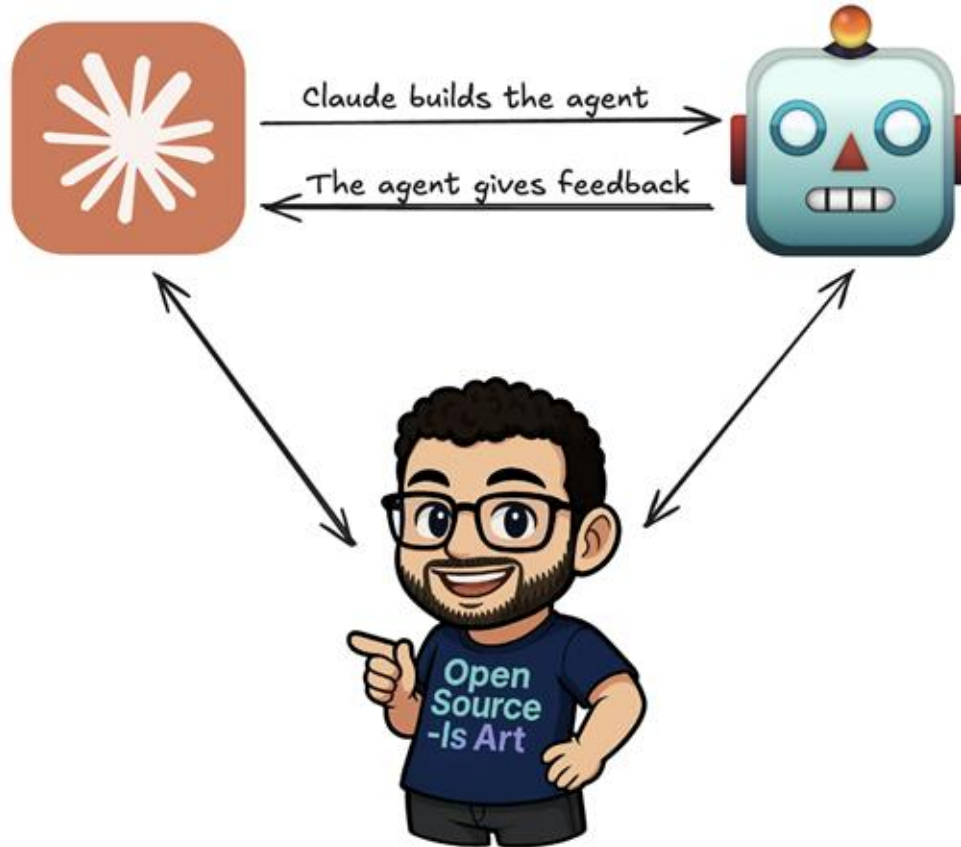


How does it works?

The core idea_



The human in the loop_



Step 1

create a job_

```
jobs > agent-2 > ↘ JOB.md > abc ## Priority Hints
1  ## Objective
2
3  <!-- What is the main goal of this optimization job? Be specific about what "better" means.
4      Examples:
5      - "Improve accuracy on the math golden dataset from ~20% to 80%+"
6      - "Reduce average latency below 500ms while maintaining current accuracy"
7      - "Add support for x questions (currently 0% pass rate on that category)"
8  -->
9
10 We want to improve the accuracy as much as we can
11
12
13 ## Target Repository
14
15 <!-- Absolute or relative path to the repo the coding agent will modify.
16 |     Also specify which branch to start from – this is the baseline. -->
17
18 - **Path**: /Users/alfonsograziano/Desktop/exp/auto-agent-demo
19 - **Branch**: master
20
21 ## Metrics
22
23 <!-- Which metric should the system optimize, and what guardrails apply to other metrics?
24 |     The primary metric determines whether a hypothesis is accepted or rejected.
25 |     Secondary constraints prevent regressions – a hypothesis is rejected if any
26 |     secondary metric regresses beyond its threshold, even if the primary metric improves. -->
27
28 - **Primary metric**: accuracy (maximize)
29 - **Secondary constraints**:
30     - latency_avg_ms: max 20% regression
31     - cost_usd: max 50% regression
32
```


Step 2.1

create baseline data_

```
export function getBaselineSystemPrompt(params: BaselineSystemPromptParams): string {
  const { targetRepoPath, baselineBranch, hypothesisDir, jobMd } = params;

  return `You are an evaluation runner for a target repository.

## Context
- Target repo: ${targetRepoPath} (branch: "${baselineBranch}")
- Report: ${hypothesisDir}/REPORT.md (update in place)

## Workflow
Read the job config, install prerequisites, run evals, update the report.

## Rules
1. Do NOT modify any files in the target repo. This is a read-only baseline run.
2. On command failure, capture the error in the report instead of retrying.
3. For each failing case, include: case id, input, expected output, actual output.
4. Fill every section in REPORT.md – use "N/A" for unavailable metrics.

## Report Fields
- **Hypothesis ID**: "000-baseline" | **Branch**: "${baselineBranch}"
- **Changes Made**: "No changes – baseline evaluation."
- **Metrics**: All values from eval output.
- **Failing Cases**: One subsection per failure (id, input, expected, actual).
- **Summary**: What works, what fails, failure patterns.
- **Recommendation**: "Baseline run – no recommendation applicable." Decision: CONTINUE

## Job Configuration
${jobMd}`;
}
```

The baseline report_

Case: fetch-country-density

- **Input:** "Using the REST Countries API... fetch Japan's population and area, then calculate the population density..."
- **Expected Output:** 326.01
- **Actual Output:** Agent refused, saying it can't access external APIs.

Case: weighted-harmonic-mean

- **Input:** "Calculate the weighted harmonic mean of Values and Weights. Round to exactly 6 decimal places."
- **Expected Output:** 29.591205
- **Actual Output:** 29.591676

Summary

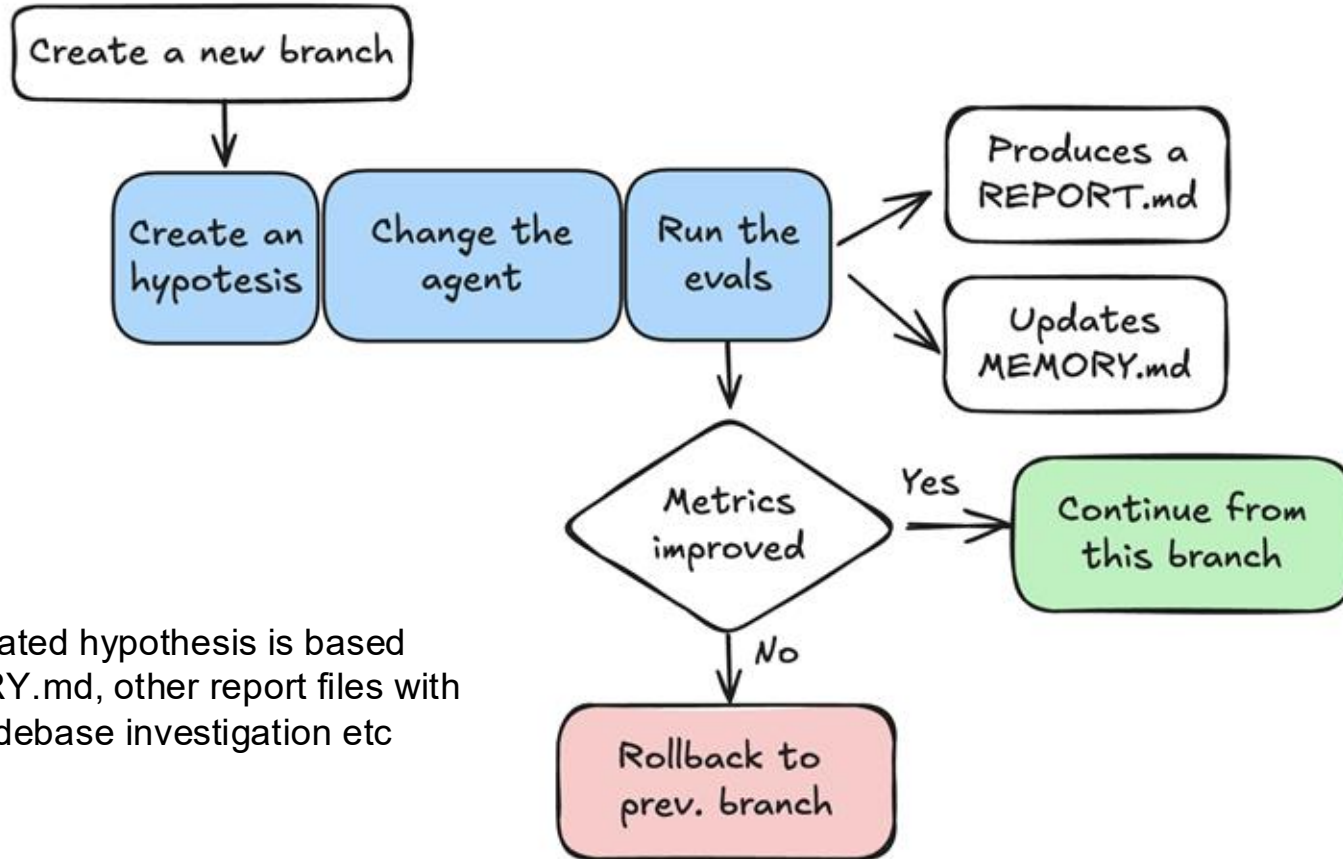
What works (11/60 passed, 18.3%):

- Simple well-known computations: trailing zeros of 200! (1), binomial coefficient C(100,50) (1), bond price (1), depreciation DDB (1), Fibonacci F(300) (1)
- Code execution tasks: FizzBuzz sum (1), staircase climbing (1), Collatz max (1)
- Some API knowledge-based answers: user count (1), user posts count (1), country borders (1)

What fails (49/60, 81.7%) — grouped by pattern:

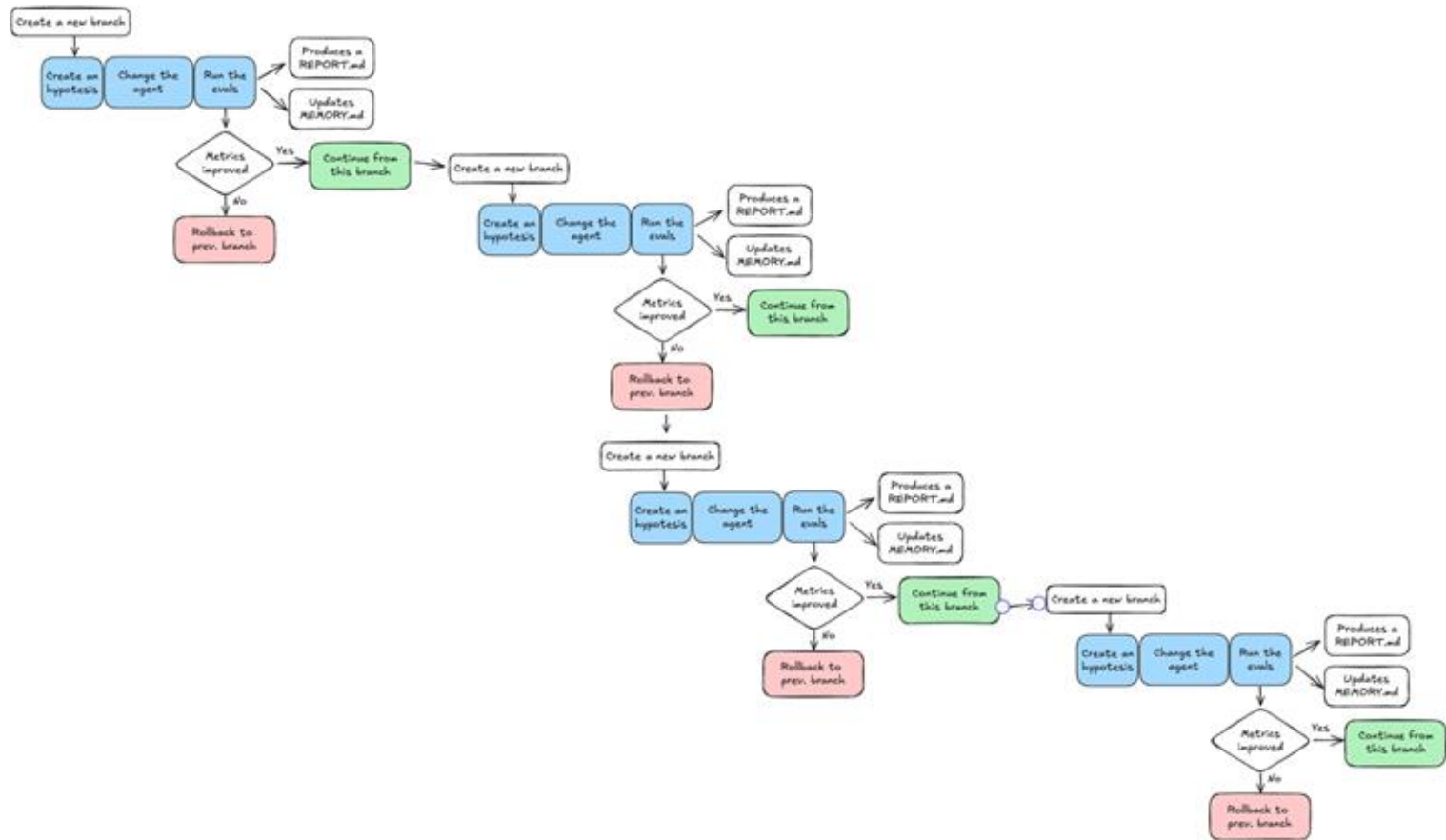
1. **Large exact integer computations (8 cases):** power-23-53, power-17-73, power-37-41, power-7-85, power-29-47, power-31-43, power-13-67, primorial-20. The agent refuses to compute large exponents, asking for a calculator tool it doesn't have.
2. **Digit-sum / modular arithmetic (3 cases):** digit-sum-88-88, digit-sum-99-99, last12-digits-7-7777. Agent either refuses or gives wrong answers.
3. **Financial math precision errors (12 cases):** fv-annuity, amortization-30yr, npv-cashflows, pv-annuity, compound-daily, continuous-compound, compound-monthly, amortization-15yr, compound-quarterly, geometric-mean, irr-cashflows, modified-duration. The agent attempts the calculation but gets slightly wrong answers due to floating-point or formula errors.

Step 2.2 - running one iteration_



*The generated hypothesis is based on MEMORY.md, other report files with failures, codebase investigation etc

Step 2.x - running every iteration_



Step 3 - generate a changelog and evaluate the results_

Changelog: agent-2

Base branch: `master` → Final branch: `agent-2-hyp-009-b31624` | Iterations: 9 (9 accepted, 0 rolled back) | Final accuracy: 83.3% (50/60)

Baseline

- **Accuracy:** 18.3% (11/60 passed)
- **Latency:** 7340 ms avg
- Agent had no tools, no sentinel value rules, and no computation capability. Failed on large integers, financial math, API fetches, and non-math classification.

Accuracy Progression

#	Hypothesis	Accuracy	Delta
000	Baseline	18.3% (11/60)	—
001	Add sentinel value rules to system prompt	31.7% (19/60)	+13.4pp
002	Add calculator tool (JS code execution)	58.3% (35/60)	+26.6pp
003	Improve calculator multi-statement handling + prompt guidance	60.0% (36/60)	+1.7pp
004	Add HTTP fetch tool	61.7% (37/60)	+1.7pp
005	Fix regex scorer for negative numbers	58.3% (35/60)	-3.4pp*

A real test on an already optimized agent_

Baseline accuracy: 76.7%

Iteration Summary:

#	Hypothesis	Decision	Accuracy
1	001-bc0b51	CONTINUE	80.7%
2	002-f7f825	CONTINUE	82.7%
3	003-5018d3	CONTINUE	81.9%
4	004-dfe15d	ROLLBACK	82.1%
5	005-a4b57e	CONTINUE	82.6%
6	006-fb02c4	CONTINUE	84.4%
7	007-81e640	ROLLBACK	81.6%
8	008-1ff45c	CONTINUE	82.9%
9	009-1889d9	CONTINUE	86.4%
10	010-691708	ROLLBACK	85.1%
11	011-0de49e	ROLLBACK	84.5%
12	012-dcb780	ROLLBACK	84.7%

- Found edge cases
- Improved the system prompt
- Improved tools description
- Fixed tools logic

Fixing bad performances on live data_



Bad
performances
on **live data**

Was this response helpful?

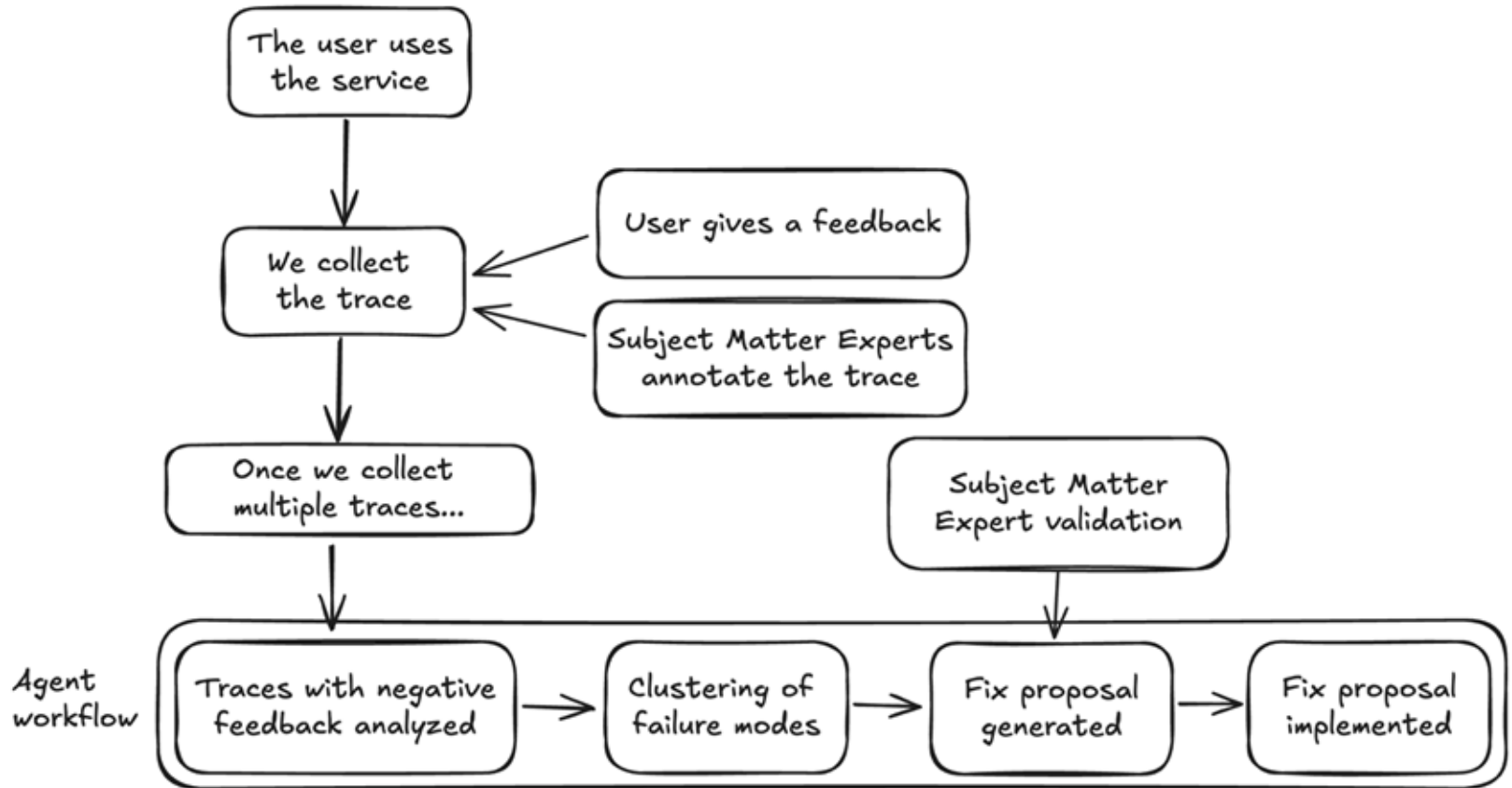
☐ YES ☐ NO

ADD A NOTE (OPTIONAL)

What could be improved?

Submit Feedback

How do we fix that?_



Step 1: The user tests the system_

The screenshot displays the 'Intelligent Monolith' AI interface. The top navigation bar includes 'Monolith AI', 'GPT-4O' (selected), 'CLAUDE 3.5', and 'DEEPSEEK'. A sidebar on the left contains a 'New Chat' button and a list of chat sessions: 'Project Alpha', 'Code Review' (selected), 'Market Research', and 'History'. At the bottom of the sidebar are 'Settings' and 'Help' buttons, and a user profile for 'Alex Thorne' with a 'UPGRADE TO PLUS' button.

The main chat area shows a user query: 'How can I optimize this React component?' (SENT AT 14:22). The AI response is titled 'OPTIMIZATION STRATEGY' and provides the following information:

To optimize your React component effectively, we should focus on reducing unnecessary re-renders and optimizing heavy computations. Based on the patterns common in modern React development, here are the primary strategies:

- 1. Memoization:** Wrap functional components in `React.memo()` to prevent re-renders unless props change.
- 2. Callback Stability:** Use the `useCallback` hook for functions passed to child components.
- 3. Derived State:** Utilize `useMemo` for expensive calculations like filtering large arrays.

Below the text, a code block titled 'OPTIMIZEDCOMPONENT.JSX' shows the following code:

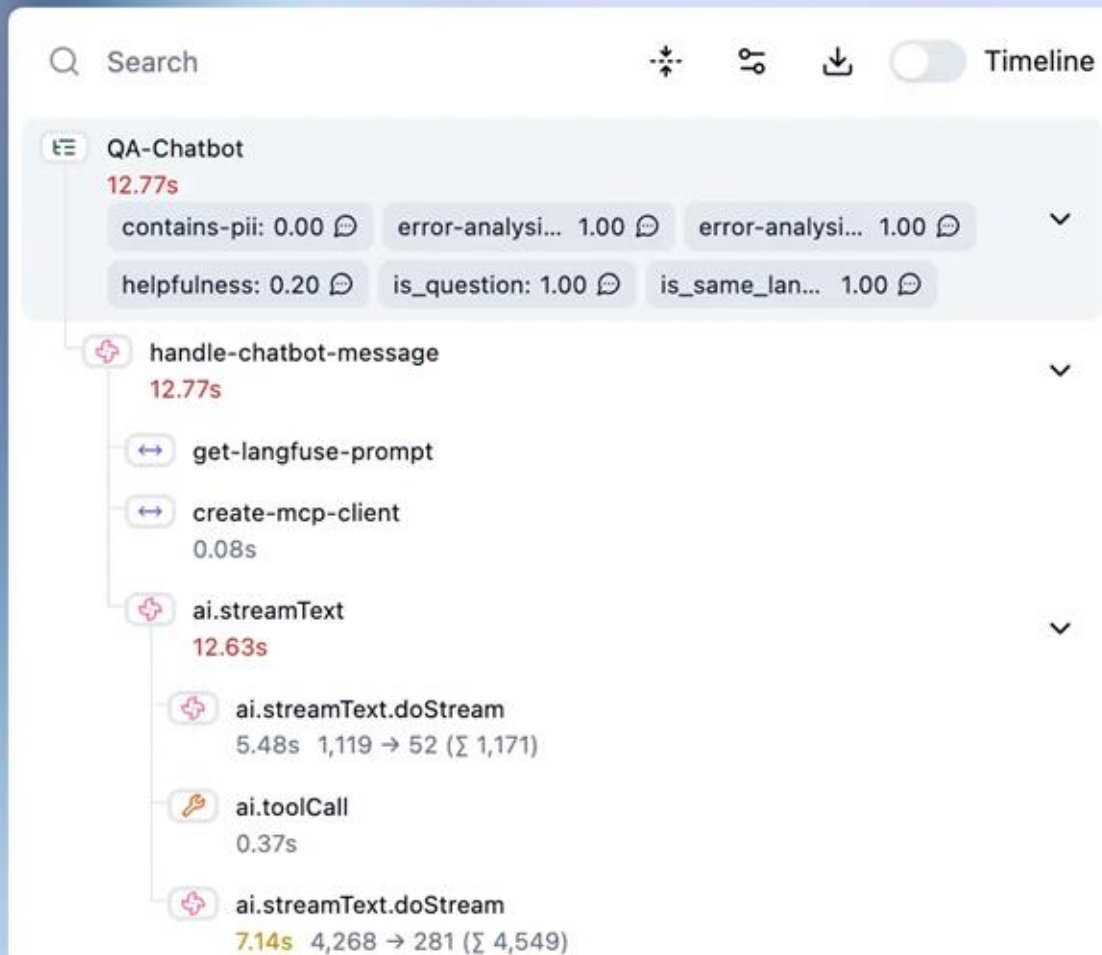
```
import React, { useMemo, useCallback } from 'react';

const OptimizedComponent = React.memo(({ data, onItemClick }) => {
  // Memoize expensive computation
  const processedData = useMemo(() => {
    return data.map(item => ({ ...item, score: item.value * 2 }));
  }, [data]);

  // Prevent function recreation on every render
```

At the bottom of the chat area is a text input field with the placeholder 'Ask anything about React, TypeScript, or Performance...' and a send button. A footer note states 'POWERED BY MONOLITH GPT-4O & 3.5 - CONTEXT: 128K TOKENS'.

We collect
tracing informations_



Step 2 (a): The user gives a feedback_

Langfuse is a platform designed to help de
models and AI applications. It provides too
behavior. This can help optimize the perfor
decisions regarding AI implementations. If



Additional Feedback

Tell us what went wrong (optional)

Very long answer

Skip Submit

Trace 1632fbb0299d5d21ab0fed2957bbf5aa

Search



Timeline



langfuse-chatbot

4.47s

user-feedba... 0.00

Very long answer

ai.streamText

4.47s

ai.streamText.doStream

4.46s 35 → 82 (Σ 117)

Step 2 (b): Subject Matter Expert annotates the trace

The screenshot displays the Langfuse web interface. The top navigation bar includes a logo, a dropdown menu set to 'PROD-EU', and links for 'Langfuse Demo', 'langfuse-docs', 'Annotation Queues', and 'Error Analysis'. The main header shows 'Queue item Error Analysis: cmj8u51fx00aoad078a2q1fx5' with 'Focused' and 'Detailed' view toggles. The left sidebar contains icons for various dashboard features. The central content area is titled 'QA-Chatbot ID' and includes a timestamp '2025-12-16 16:59:55.570'. It displays session and user IDs, an environment of 'default', and the input text 'What can I use Langfuse for?'. The output section shows a green-highlighted response about Langfuse's capabilities. On the right, an 'Annotate' panel is highlighted with a red border, featuring a 'Score data saved' status, a 'First Pass' label, and a 'Good Output' dropdown menu. At the bottom, navigation controls show '1 / 3', 'Back', 'Skip', and a 'Complete + Next' button.

PROD-EU Langfuse Demo / langfuse-docs / Annotation Queues / Error Analysis

Queue item Error Analysis: cmj8u51fx00aoad078a2q1fx5 Focused Detailed

QA-Chatbot ID + Add to datasets

2025-12-16 16:59:55.570

Session: chat_fb935e47-8e2b-4c48-85b3-62f7a7289f87 User ID: u-mgzMxSoG2E3afI9G9ZF3V

Env: default

Input

What can I use Langfuse for?

Output

What you can use Langfuse for

Langfuse is an open-source LLM engineering platform to build, debug, and improve LLM apps end-to-end.

- Observability & tracing: capture prompts, model responses, token/cost, latency, tool/RAG steps, sessions, and agent graphs. See the overview and quickstart: [Observability & Tracing](#) · [Get started](#) · [Demo](#)
- Prompt management: version, label, deploy prompts, test in a playground, and link prompt versions to traces. See [Prompt Management overview](#) · [Get started](#)
- Evaluation: run LLM-as-a-judge, collect user feedback, annotate data, build datasets, and run experiments to compare models/prompts. See [Evaluation overview](#) · [Experiments](#) · [Datasets](#)
- Monitor & evaluate: monitor quality, cost, latency, usage over time, export or import via API. See

Annotate ✓ Score data saved

First Pass Good Output X

1 / 3 ← Back Skip → Complete + Next

Step 3: Collect all the traces with feedback locally_

```
npm run fetch-traces-with-feedback.ts --from 2026-03-30 --to 2026-04-03 --limit 200
```

```
1  {
2    "fetchAt": "2026-04-03T12:24:50.619Z",
3    "environment": "production",
4    "totalScanned": 183,
5    "totalWithFeedback": 114,
6    "traces": [
7      {
8        "id": "f47ac10b928e4d27b11a92837d8e921d",
9        "userId": "j.smith@quantum-solutions.io",
10       "timestamp": "2026-04-05T08:14:12.115Z",
11       "model": "us.anthropic.claude-sonnet-4-5-20250929-v1:0",
12       "latency": 4.12,
13       "question": "What is the recommended torque specification for the titanium alloy bolts on the CX-500 engine components?",
14       "answer": "To find the specific torque requirements for the CX-500 engine components, please refer to the manufacturer's technical specifications. The recommended torque for the titanium alloy bolts is 18 Nm (13.3 ft-lb).",
15       "comments": [],
16       "userFeedback": {
17         "score": 1,
18         "comment": "Solid guidance, but could be more precise. 7/10. \n1) You should have linked directly to the CX-500 engine manual."
19       },
20       "annotations": []
21     ]
22  }
```



Step 4: Run the analysis_

```
agents > skills > trace-feedback-analysis > SKILLmd > # Trace Feedback Analysis Skill > ## Report Format
1 ---
2 name: trace-feedback-analysis
3 description: "Analyze Langfuse trace feedback data to generate a structured failure-mode report. Use this skill when asked to analyze agent traces, user feedback, or generate a feedback analysis report. Requires a JSON traces file as input."
4 metadata:
5   author: Nearform
6   version: "1.0.0"
7 ---
8
9 # Trace Feedback Analysis Skill
10
11 ## Purpose
12
13 Analyze a Langfuse traces-with-feedback JSON file and produce a structured failure-mode report that engineering teams can use to prioritize system improvements.
14
15 ---
16
17 ## Prerequisites - Input File
18
19 This skill requires a **traces-with-feedback JSON file** as input. The user MUST provide the filename.
20
21 ### If no filename is provided
22
23 Respond with:
24
25 > To generate a feedback analysis report I need the path to a traces-with-feedback JSON file.
26 >
27 > These files live in "scripts/langfuse-dataset-run/output/" and are named like "traces-with-feedback-<timestamp>.json".
28 >
29 > If no file exists yet, I can create one by running:
30 > ---
31 > cd scripts && npm run langfuse:traces:feedback -- --limit 100
32 > ---
33 > This fetches the latest 100 traces with human feedback from Langfuse and writes a JSON file to "scripts/langfuse-dataset-run/output/".
34 >
```

Cluster failures
based on feedback



Do root cause analysis



Propose possible
improvements



Generate a full report

Report example (1/2)

Executive Summary

Metric	Value
Total traces scanned	400
Traces with feedback	100
Positive feedback (score = 1)	60
Negative feedback (score = 0)	40
Negative rate	40%
Traces with comments (score = 1)	28
Traces with comments (score = 0)	38
Traces with annotations	6

The analysis reveals 10 distinct failure mode clusters. The top three by frequency are:

1. Wrong product/page recommended (incorrect routing) — 13 instances
2. Missing pages in the sitemap / knowledge base gaps — 9 instances
3. Response too long / information overload — 9 instances

Cluster 5: URLs Not Hyperlinked / Formatting Inconsistencies

Frequency: 10+ traces | Severity: MEDIUM | Impact: Users can't click links, poor experience

This is one of the most consistently reported issues across all testers. URLs appear as plain text instead of clickable hyperlinks, and formatting varies wildly between responses.

Trace	User Feedback
1101c3e8	"The links are not hyperlinked"
fcdd1925	"Links are not hyperlinked"
24b78720	"Recommended pages was not hyperlinked"
1661ddc3	"Link was not hyperlinked, so I could not click on it directly"

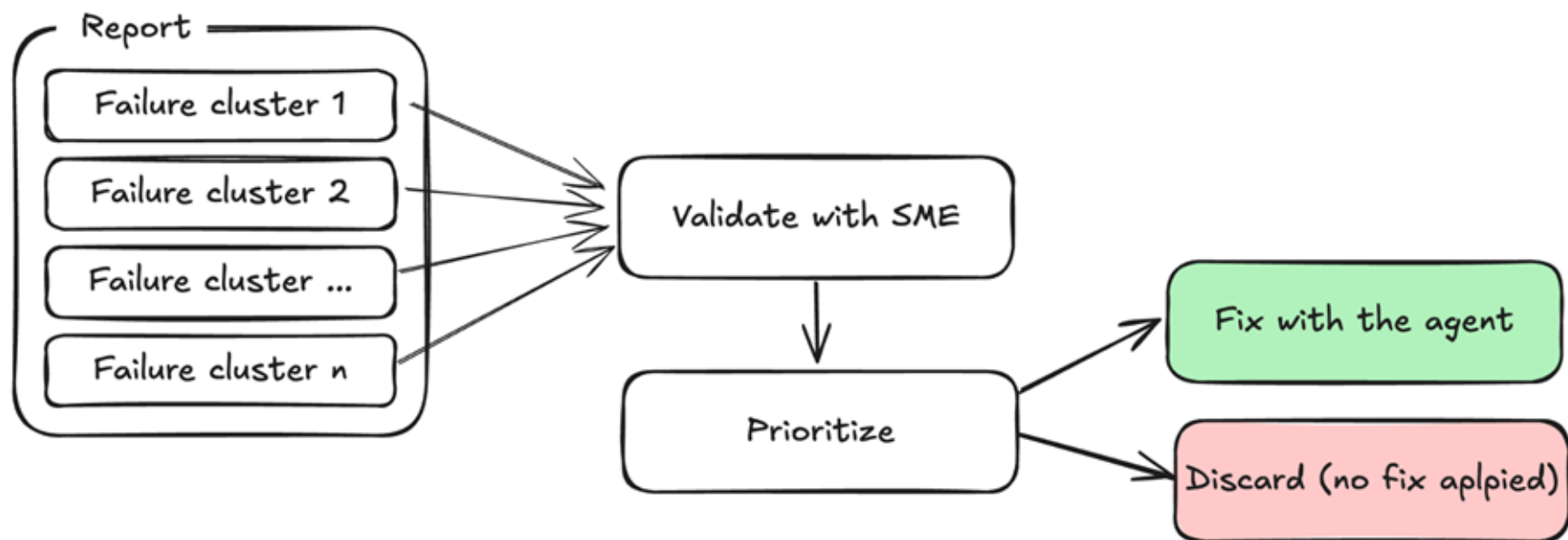
Root Causes:

- Some models/responses use markdown link syntax, others use plain URLs
- The UI rendering may not auto-link plain URLs
- Inconsistent prompt instructions across models

Recommendations:

1. Enforce markdown link format in all responses: `[Label] (URL)` — never bare URLs
2. Add a post-processing formatter that converts any bare URL to a markdown link
3. Standardize response templates across all models (Claude, GPT, Kimi, DeepSeek)
4. Test link rendering in the UI for each model's output format

Step 5: Prioritize and act on the feedback_



Failure modes found during this investigation step become part of the golden dataset, and the evals suite is updated to spot regressions.

This report gets generated at least once per sprint.

In multiple cases a coding agent has been able to fix an entire suite of issues zero shot.

Complex cases might require the usage of auto-agent

All this is possible, thanks to

Harness Engineering

<https://martinfowler.com/articles/harness-engineering.html>
<https://openai.com/index/harness-engineering/>

Harness Engineering refers to the idea of building the environment around AI coding agents so they can work reliably: constraints, tests, feedback loops, and governance.

Spec-Driven Development

A methodology which allows us to create specs which define our intention, constraints, functional requirements and an implementation plan for the agent.

Quality gates

Unit tests on the codebase to catch regressions early, a CI/CD pipeline, regression evals, linting, security scanning and much more - to catch issues soon!

Context engineering

Every repeated workflow becomes a skill. Important things go in rules files or CLAUDE.md files. Creating context becomes the key to continuous improvement.

Observability

If we don't know what happens when we ship in production, we're blind. Proper observability in place allows us to quickly catch issues and debug properly.



Thank you!_